Langage Interprété

Structures de données et écriture idiomatique

1ère année Cycle ingénieur ISEN – Lille

2026/2027

Abderrahmane Kasmi

Séquence 2

Structures de données et écriture idiomatique

01 **Collections natives**

02 **Compréhensions**

03 **Fonctions de haut niveau (map, filter, sorted)**

04 **Itérateurs et générateurs (yield)**

05 **Ecriture idiomatique**

01

Collections natives

Les listes

Une liste (`list`) est une structure de données qui permet de stocker plusieurs éléments dans un **ordre défini**. Elle peut contenir des **éléments de types différents**

```
fruits = ["pomme", "banane", "orange"]

fruits.append("kiwi")
fruits.remove("banane")
fruits[0] = "fraise"
```

- Les listes sont des structures mutables

Les tuples

Un tuple (`tuple`) est une structure de données qui permet de stocker plusieurs éléments dans un **ordre défini**. Contrairement à une liste, un tuple est **immuable**

```
coordonnees = (48.86, 2.35)
```

- Les tuples sont des structures immutables
- Un tuple est hashable uniquement si tous ses éléments sont eux-mêmes hashables

Les tuples

- **Retourner plusieurs valeurs depuis une fonction**

```python
def calculer(a, b):
    return a + b, a * b

resultat = calculer(3, 4)

print(resultat)
# (7, 12)
```

Les ensembles

Un ensemble (`set`) est une structure de données qui permet de stocker une collection d'éléments **uniques et non ordonnés**

```
nombres = {1, 2, 3}
nombres.add(4)
nombres.remove(2)
```

- Les ensembles sont des structures mutables
- Les ensembles n’acceptent que les éléments hashables

Les ensembles

- **Utiliser les valeurs de vérité**

```python
nombres = [1, 2, 2, 3, 4, 4, 5]

nombres_uniques = set(nombres)

print(nombres_uniques)
```

- **Opérations mathématiques sur les ensembles**

```python
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)  # union
print(A & B)  # intersection
print(A - B)  # différence
```

Les dictionnaires

Un dictionnaire (`dict`) est une structure de données qui permet de stocker des informations sous forme de paires **clé–valeur**. Chaque clé est unique et permet d'accéder **rapidement** à la valeur qui lui est associée

```
personne = {
    "nom": "Alice",
    "age": 25,
    "ville": "Paris"
}
```

- Les dictionnaires sont des structures mutables
- Les ensembles n’acceptent que des clés hashables
- Les valeurs peuvent être de n'importe quel type

02

Compréhensions

Les compréhensions

Une compréhension construit une collection à partir d'une autre en **une seule expression**. C'est la marque de fabrique du style Python

- Elle remplace le process: créer une liste vide → boucler → ajouter des éléments.

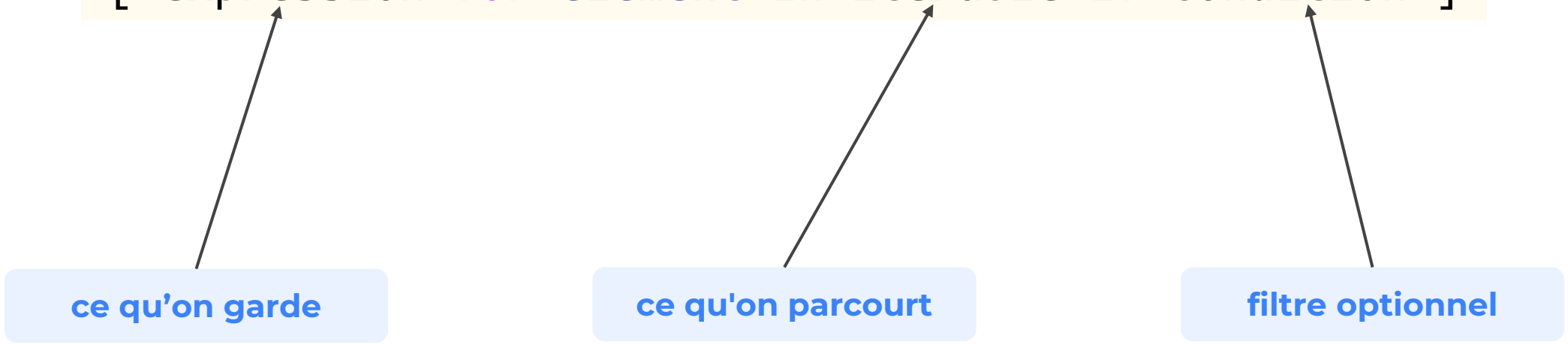
Compréhension de liste

```python
carres = []
for n in range(10):
    carres.append(n ** 2)
```

```python
carres = [n ** 2 for n in range(10)]
```

Compréhension de liste

```
[ expression for élément in itérable if condition ]
```



Compréhension de liste

```
notes = [12, 15, 8, 17, 9]


admis      = [n for n in notes if n >= 10]
bonus    = [min(n + 2, 20) for n in notes]
mentions    = ["admis" if n >= 10 else "ajourné" for n in notes]
```

Compréhension de liste

```python
noms = ["Dupont", "Martin", "Bernard"]
notes = [13.5, 9.0, 16.25]

# dict : {clé: valeur for ...}
carnet = {nom: note for nom, note in zip(noms, notes)}

# Inverser un dictionnaire
inverse = {v: k for k, v in carnet.items()}

# Filtrer un dictionnaire
recus = {nom: n for nom, n in carnet.items() if n >= 10}

# set : pas de doublons
initiales = {nom[0] for nom in noms}
```

03

Fonctions de haut niveau

Lambda

Une fonction **anonyme**, avec **une seule expression**

```
lambda paramètres: expression
```

```
carre = lambda x: x ** 2

print(carre(5))
```

- Les fonctions lambda sont utilisées seulement en cas de fonction simple
- Les fonctions lambda acceptent plusieurs paramètres

Map

Une fonction qui effectue **la même opération** sur les éléments d'une collection

```
def myfunc(n):
  return len(n)

x = map(myfunc, ('apple', 'banana', 'cherry'))

print(list(x))
```

Filter

Une fonction qui sélectionne les éléments d'une collection qui respectent **une condition donnée**

```
ages = [5, 12, 17, 18, 24, 32]

def myFunc(x):
  if x < 18:
    return False
  else:
    return True

adults = filter(myFunc, ages)
```

Sorted

Une fonction qui ordonne les éléments selon **un critère donné**

```
def myfunc(n):
  return abs(10-n)



a = (5, 3, 1, 11, 2, 12, 17)
x = sorted(a, key=myfunc)
print(x)
```

04

Itérateurs et générateurs

Les itérables

Un itérable est un objet Python dont **on peut parcourir les éléments un par un**, par exemple avec une boucle for

- **Listes, Dictionnaires, Tuples, Ensembles**
- **Chaines de caractères**
- **`Range()`**

Les itérateurs

Un itérateur est un objet qui permet de **parcourir les éléments d'un itérable un par un**. Il mémorise où il en est dans le parcours et fournit l'élément suivant

```
nombres = [10, 20, 30]

it = iter(nombres)

print(next(it))
print(next(it))
print(next(it))
```

Les itérateurs

```python
nombres = [10, 20, 30]

for nombre in nombres:
    print(nombre)
```

```python
nombres = [10, 20, 30]

it = iter(nombres)

while True:
    try:
        nombre = next(it)
        print(nombre)
    except StopIteration:
        break
```

Les générateurs

Un générateur est un type particulier d'itérateur qui **produit ses valeurs une par une**, au fur et à mesure qu'elles sont demandées, au lieu de les stocker toutes en mémoire

```
def nombres():
    yield 1
    yield 2
    yield 3

g = nombres()

print(next(g))
print(next(g))
print(next(g))
```

Les générateurs

```python
def nombres():
    for i in range(5):
        yield i
```

```python
for nombre in nombres():
    print(nombre)
```

Tout générateur est un itérateur

05

Ecriture idiomatique

Idiomes à connaître

- **Parcourir une collection**

```python
names = ["Alice", "Bob", "Charlie"]

for i in range(len(names)):
    print(names[i])
```

```python
names = ["Alice", "Bob", "Charlie"]

for name in names:
    print(name)
```

- **Utiliser les compréhensions**

```python
squares = []

for x in range(10):
    squares.append(x ** 2)
```

```python
squares = [x ** 2 for x in range(10)]
```

Idiomes à connaître

- **Utiliser `enumerate()`**

```python
names = ["Alice", "Bob", "Charlie"]

for i in range(len(names)):
    print(i, names[i])
```

```python
names = ["Alice", "Bob", "Charlie"]

for i, name in enumerate(names):
    print(i, name)
```

- **Décomposer les structures**

```python
point = (10, 20)

x = point[0]
y = point[1]
```

```python
point = (10, 20)

x, y = point
```

Idiomes à connaître

- **Chercher un élément dans une collection**

```python
for name in names:
    if name == "Alice":
```

```python
if "Alice" in names:
```

- **Utiliser les valeurs de vérité**

```python
if len(names) > 0:
    print("La liste n'est pas vide")
```

```python
if names:
    print("La liste n'est pas vide")
```

Idiomes à connaître

- **Utiliser `dict.get()`**

```python
if "age" in person:
    age = person["age"]
else:
    age = 0
```

```python
age = person.get("age", 0)
```

- **Utiliser `with` pour gérer les ressources**

```python
file = open("data.txt")

content = file.read()

file.close()
```

```python
with open("data.txt") as file:
    content = file.read()
```